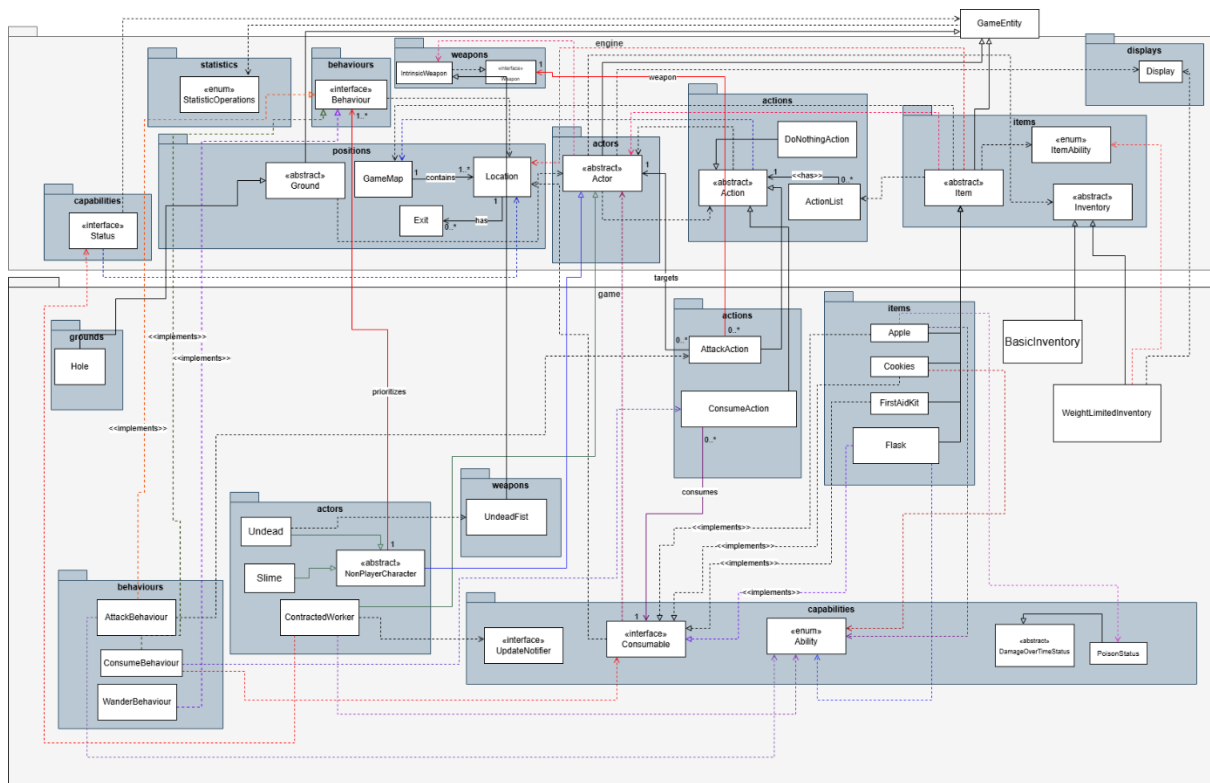
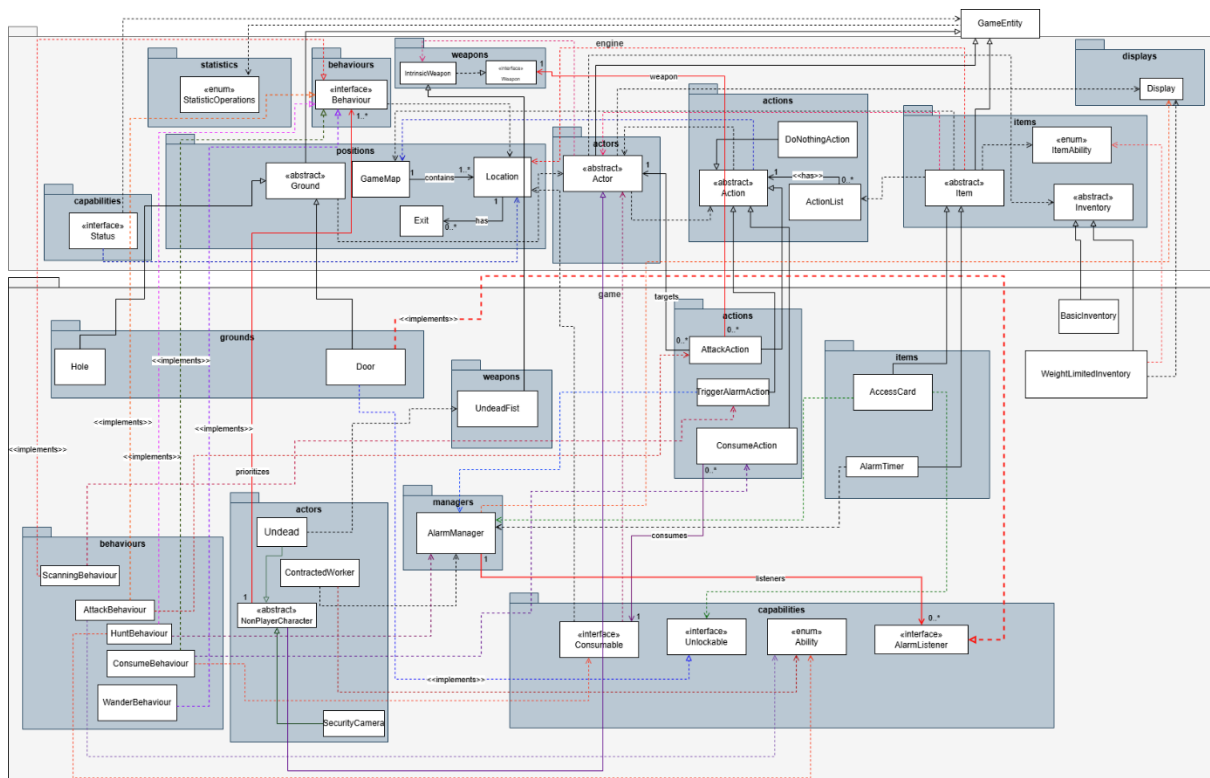


Design Diagrams

FIT2099 Assignment 1 Design Documentation



REQ 4 UML Class Diagram:



Design Rationale

REQ1 - Garbage Collection Inc.

Design 1: Create specific, concrete action classes for every individual item interaction (e.g., ConsumeFlaskAction, UseFirstAidKitAction).

Pros	Cons
Simple Implementation: Easy to understand for a single item without needing to create interfaces or abstract methods.	Code Duplication (DRY Violation): Multiple classes would contain nearly identical logic for finding targets or executing effects, leading to maintenance issues.
No Abstraction Overhead: Avoids the initial complexity of setting up interfaces like Consumable or Unlockable.	Low Extensibility: Adding a new item requires creating a brand-new Action class, cluttering the actions package and increasing coupling.
	Violation of OCP: The ContractedWorker class would need to be modified with specific logic to handle each new action type.

Design 2: Implement generalized interaction interfaces (Consumable and Unlockable) used by a single ConsumeAction and UnlockDoorAction.

Pros	Cons
Promotes Code Reuse (DRY): A single ConsumeAction can handle any item that fulfills the Consumable contract, regardless of its specific effects.	Increased Complexity: Requires setting up an interface-based architecture, which involves more initial files and planning.
High Extensibility (OCP): New scrap items can be added to the moon by simply implementing Consumable. The existing action classes remain "closed" for modification.	Indirect Logic: Developers must check the Item class to understand the outcome of an action, rather than seeing it directly in the Action class.
Decoupling (DIP): High-level modules like ConsumeAction do not depend on low-level concrete items like Flask. Both depend on the Consumable abstraction.	
This design also respects the Interface Segregation Principle (ISP) because only items that need consumption behavior implement the Consumable interface. Items like AccessCard are not forced to depend on methods they will never use.	

of the weight limit is hidden within the add() and remove() methods, ensuring that the ContractedWorker cannot enter an illegal state by carrying too much scrap. I maintained literal values for item weights where the ItemStatistics.WEIGHT Enum provides immediate semantic context, adhering to KISS by avoiding 'constant pollution' where the number's purpose is already clear from the code structure.

Hypothetical Extension Example: If a future requirement introduced a new consumable item such as a "Energy Drink," the developer would only need to implement the Consumable interface on the new class and define its consumedBy() method. The existing ConsumeAction would handle the interaction automatically, and the ContractedWorker would require no modifications. This demonstrates how the design supports open-closed principle in practice.

Trade-off Analysis: The primary trade-off in Design 2 is upfront abstraction overhead versus long-term code duplication prevention. The costs include creating four new files (two interfaces, two action classes) and requiring developers to understand interface-based polymorphism. However, without this abstraction, each item would require its own action class, resulting in approximately ten redundant action classes across the full assignment. The abstraction overhead is a one-time cost, while code duplication would accumulate with each new requirement.

Finally, the AccessCard uses Polymorphism to interact with any Ground that implements the Unlockable interface. In REQ1, this is used for the ship's Door. By depending on an abstraction rather than a concrete class (Dependency Inversion Principle), the card is automatically compatible with any future lockable objects introduced to the Moon's facility without requiring code changes to the AccessCard logic.

REQ2: Starter Moon: 99-Deprecated

A. Management of Recurrent Damage (Status Effects)

Design 1: Implement recurrent damage logic (poison/burning timers) directly within each concrete Item and Ground class (e.g., Apple and Puddle classes tracking their own damage turns).

Pros	Cons
Simpler to implement for a single item without creating an abstraction layer.	Violation of DRY: The logic for "decrement health by 1 per turn for X turns" would be duplicated across Apple, Cookies, Puddle, and Fire classes.
No need for an external Status system.	Maintenance Difficulty: If the damage-per-turn value needs to change from 1 to 2, every individual class must be manually updated.

	Violation of SRP: Items would be responsible for both their identity and the temporal logic of an actor's internal biological state.
--	--

Design 2: Create an abstract `DamageOverTimeStatus` class that utilizes the engine's status ticking system, with specific concrete subclasses like `PoisonStatus` and `BurningStatus`.

Pros	Cons
Promotes Code Reuse (DRY): All turn-based decrement logic is centralized in the abstract parent class. Concrete statuses only define their name and duration.	Requires a slightly more complex architecture involving an abstract class and inheritance.
High Extensibility (OCP): New status effects, such as "Radiation," can be added by creating a new subclass without modifying existing item logic.	Indirect logic flow: The damage application is separated from the item that triggered it.
Consistent Behavior: Ensures all "poisoned" effects in the game behave identically regardless of whether they came from an Apple or a Puddle.	
To improve maintainability, I replaced magic numbers with symbolic constants for all balance-related values (e.g., heal points and durations). Following the Single Point of Truth (SPOT), these are linked to UI messages via <code>String.format</code> , ensuring that logical updates automatically reflect in player feedback and preventing data inconsistencies."	

- Design 2 was chosen over Design 1 despite requiring an abstract class and inheritance hierarchy. The trade-off of increased architectural complexity is justified because REQ2 introduces four separate damage sources (Apple, Cookies, Puddle, Fire). Without centralization, poison logic would be duplicated across all four classes, violating DRY and making maintenance difficult.

B. Implementation of Fire Overlapping (Stacking Logic)

Design 1: Use a single integer variable in the `Fire` class to track duration. If a new leak occurs on an existing fire tile, the timer simply resets to 5.

Pros	Cons
Low memory usage and very simple mathematical implementation.	Violation of REQ2: Fails to scale damage when multiple leaks occur, violating the rule that damage should increase if a lantern leaks twice.

	Weak Reality Modeling: Does not accurately reflect the intensity of multiple fire sources on a single coordinate.
--	---

Design 2: Implement the FireStackable interface and use a List<Integer> stacks within the Fire class to track individual fire instances independently.

Pros	Cons
High Cohesion: The Fire class is solely responsible for managing the intensity and lifespan of the hazard.	Higher memory overhead as a List object must be maintained for every fire tile.
Strict Requirement Alignment: Allows for damage scaling. If two fires exist, the tick method adds two BurningStatus effects, satisfying "2 hp per turn".	Requires more complex iteration logic during the tick() method to remove expired stacks.

- Design 2 (List<Integer>) was chosen over Design 1 (single integer) despite higher memory overhead per fire tile. The trade-off of storing a List object is acceptable because the game map contains at most 2000 tiles, and the memory cost is negligible. The benefit of correctly implementing stacking damage (2 HP per turn for overlapping fires) is required by the specification and cannot be achieved with Design 1.

C. Background Turn Notifications (Lantern Leak Messages)

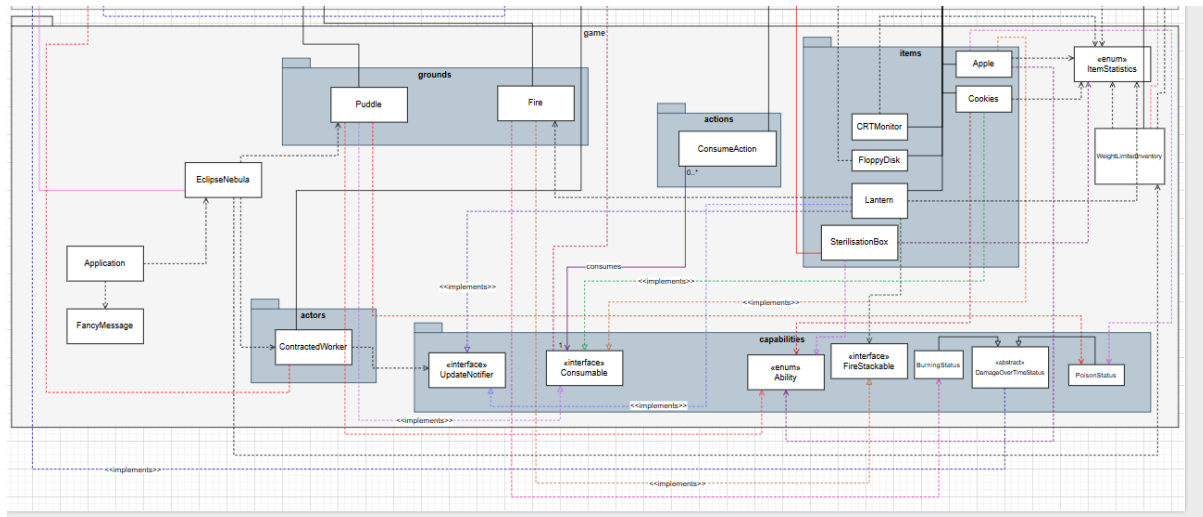
Design 1: Have the Lantern class directly access the Display object to print leak messages during its tick() method.

Pros	Cons
Immediate feedback to the user without needing intermediate interfaces.	Tight Coupling: The Lantern class becomes dependent on the Display class, making it harder to test or modify the UI.
	Violation of SRP: Items should manage their state. The Actor or World should manage what is printed to the console.

Design 2: Create an UpdateNotifier interface that allows the Lantern to store a message, which is then retrieved and printed by the ContractedWorker during its playTurn().

Pros	Cons
Decoupling: The Lantern does not know how or where the message is printed, maintaining a clean separation between logic and UI.	Requires the ContractedWorker to actively poll inventory items for messages each turn.
Interface Segregation (ISP): Only items that actually need to notify the user (like the Lantern) implement the interface.	

- Design 2 was chosen over direct Display access despite requiring the ContractedWorker to poll inventory items each turn. The trade-off of slight runtime overhead is justified by the decoupling benefit - the Lantern class no longer depends on the Display class, making it testable and reusable if the UI system changes in the future.



The diagram above shows the final design of requirement 2, which utilizes Design 2 for parts A, B, and C. While Design 2 introduces abstractions, it follows KISS by centralizing all damage logic into one `DamageOverTimeStatus` class, keeping each item simple and focused on its primary responsibility.

I utilized Polymorphism via the `Consumable` interface. This allows the `Puddle` (a `Ground` type) and the `Apple` (an `Item` type) to be processed by the same `ConsumeAction`. This follows the Dependency Inversion Principle (DIP) because the high-level `Action` module depends on an abstraction rather than the concrete `Puddle` or `Apple` classes.

The `Fire` class implementation follows KISS by encapsulating all stacking complexity within a single class. External classes simply call `addStack()` without needing internal implementation details. This satisfies SRP as the `Fire` class handles hazard intensity while `BurningStatus` handles health reduction. This approach ensures that if a worker stands on two overlapping fires, they receive 2 damage per turn.

The `UpdateNotifier` interface follows Interface Segregation Principle (ISP) because only the `Lantern` implements it. Items that never need to print messages are not forced to depend on this interface.

The interaction with the `SterilisationBox` is handled via the Open-Closed Principle (OCP). Instead of hardcoding the box into every food item, the items check if the consumer has the `Ability.STERILIZER`. This allows the system to be "open" to new sterilization items in the future without modifying the logic of the `Apple` or `Cookies`.

The DamageOverTimeStatus hierarchy demonstrates Liskov Substitution Principle (LSP). Any status subclass (Poison or Burning) can be added to an Actor's status list and processed by the engine's ticking logic without the engine needing to know the specific type of damage being applied. Changing poison damage from 1 to 3 requires modifying only PoisonStatus, not the five separate classes that apply poison.

REQ3: The Dark Side of the Moon

A. Actor Decision-Making Logic (Strategy Pattern)

Design 1: Implement the logic for attacking, wandering, and consuming directly within the playTurn() method of the Slime and Undead classes using conditional if-else blocks.

Pros	Cons
Simple implementation for actors with very limited logic.	High Code Duplication (Violation of DRY): The "Wander" logic would be repeated in every creature class.
No need for additional behaviour classes or interface overhead.	Low Extensibility: Adding a new logic (e.g., "Fleeing") requires modifying the core code of every actor.
	Violation of SRP: The Actor class becomes responsible for both its state and the complex algorithms of its decision-making.

Design 2: Implement a Strategy Pattern using the Behaviour interface and a TreeMap within each actor to store and prioritize automated actions.

Pros	Cons
High Code Reuse (DRY): WanderBehaviour is written once and shared by Workers, Slimes, and Undead.	Increased complexity: Requires managing a collection of behaviours and understanding their priority weights.
Strict Priority Management: The TreeMap ensures that a Slime will always prioritize "Eating" over "Wandering" if both are possible.	Requires more initial boilerplate code to set up the mapping system.
Separation of Concerns (SRP): The NonPlayerCharacter base class manages turn-taking logic, while subclasses manage identity and specific behavior composition.	

B. Creature Spawning Mechanism

Design 1: Create a global "CreatureManager" class that iterates through the map each turn to identify hole coordinates and decide when to spawn a creature.

Pros	Cons
------	------

Centralized control over the total number of creatures in the world.	Tight Coupling: The manager would need intimate knowledge of map coordinates and the internal state of every "Hole" tile.
	Inefficient: Iterating over the entire map every turn to find holes is computationally more expensive than localized logic.

Design 2: Override the tick() method within the Hole ground class to manage localized counters and spawning logic for that specific coordinate.

Pros	Cons
Encapsulation: Each hole manages its own 20-turn counter independently, preventing a single global state from failing.	Harder to implement world-wide spawning caps if needed in the future.
Efficiency: The engine naturally calls the tick() of Ground objects, meaning spawning logic only executes for tiles that are actually holes.	

C. Slime gluttony and item interaction (Direct Consumption)

Design 1: Require the Slime to perform a PickupAction first, then a ConsumeAction on the next turn, mirroring the player's interaction with inventory.

Pros	Cons
Reuses the player's existing interaction model.	Violation of REQ3 (Edit 07/04): The requirement specifically states slimes consume directly from the ground without inventory use.
	Unnecessary Logic: Slimes do not need inventory weight management. Forcing it adds unnecessary code.

Design 2: Implement a ConsumeBehaviour that checks the Location for Consumable items and calls consumedBy() directly from the ground.

Pros	Cons
Strict Requirement Alignment: Perfectly satisfies the gluttonous nature of the slime by allowing immediate consumption from the ground.	Increased Lifecycle Complexity: Items must manage their own ground-based removal or persistence logic, as the standard inventory cleanup system is bypassed.
Polymorphism: The same ConsumeAction logic works for slimes on the ground and workers using their inventory.	State Dependency: The behaviour depends on items providing accurate allowableActions. An oversight here

	could cause actors to waste turns on non-functional items.
--	--

- NonPlayerCharacter logic enables the Slime to utilize ConsumeBehaviour to validate item usability via allowableActions(). It skips non-functional items (e.g., empty Flasks or Kits on cooldown) with a notification, while consuming functional ones. Items persist on the ground even after Slime interaction, ensuring these corporate assets remain available for workers to reclaim (REQ1), only consuming Apple/Cookies it will call the cleanUp() method to remove from the ground.

D. Behaviour Priority Management (HashMap vs TreeMap)

Design 1: Use HashMap to store creature behaviours with integer priorities.

Pros	Cons
Simple implementation with no ordering overhead.	HashMap does not guarantee iteration order, causing inconsistent behaviour execution priority.
Efficient constant-time get and put operations.	If two behaviours have the same priority, execution order becomes unpredictable.

Design 2: Use TreeMap to store creature behaviours with integer priorities.

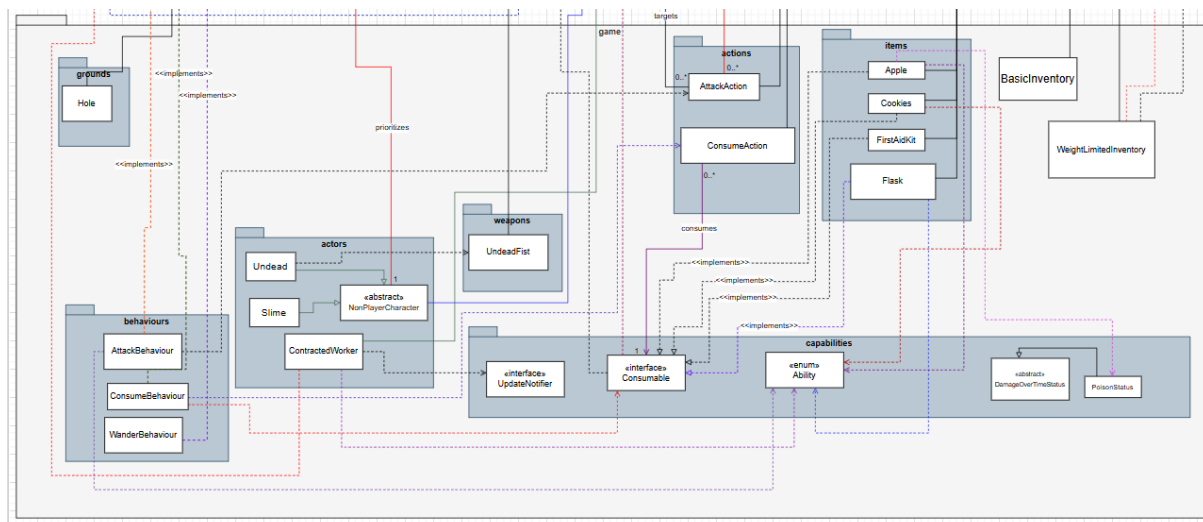
Pros	Cons
Maintains natural ordering by priority key, ensuring AttackBehaviour always executes before WanderBehaviour.	Slower $O(\log n)$ performance compared to HashMap's $O(1)$.
Clear and predictable behaviour execution every turn.	Slightly higher memory overhead to maintain tree structure.

- TreeMap was chosen over HashMap despite the performance trade-off. The benefit of guaranteed priority ordering ensures that Undead always attacks before wandering, which is critical for satisfying the requirement that hostile creatures prioritize combat over movement.

E. Weapon Architecture (Modularity)

Create a dedicated UndeadFist concrete class to encapsulate weapon properties.

Pros	Cons
High Cohesion/SRP: Strictly separates the actor's behavioral logic from its physical weapon properties.	Slight Abstraction Overhead: Requires managing an additional file in the weapons package.
Improved Readability: Removes cluttered anonymous logic ({}) from the Actor class, following engine demo conventions.	



The diagram above shows the final design of requirement 3, which utilizes Design 2 for parts A, B, and C.

To implement the Undead and Slime, I utilized a NonPlayerCharacter abstract base class to centralize the Strategy Pattern. This follows the Open-Closed Principle (OCP); new 'Hunting' or 'Scanning' behaviours can be added via composition without modifying the base turn-taking loop. By managing the TreeMap within this parent class, logic priorities are handled consistently across all automated entities.

This design follows DRY by centralizing the playTurn loop within NonPlayerCharacter, allowing Slime, Undead, and SecurityCamera to share a unified decision-making core. It satisfies LSP, as any Behaviour subclass can be substituted into the NPC's behavior map without altering core logic. ISP is respected as actors only compose the specific behaviours they require. For example, Slime implements ConsumeBehaviour but Undead does not. If a new "Scavenger" creature were added in a future requirement, it could reuse WanderBehaviour and ConsumeBehaviour immediately. The trade-off of increased boilerplate code (TreeMap initialization) is justified by the ability to add new behaviours without modifying existing actor classes.

The Hole class uses the engine's tick(Location) hook to implement spawning. This adheres to the Single Responsibility Principle (SRP), as the ground tile itself manages the 20-turn spawn cycle. By checking !location.containsAnActor() before spawning, the design prevents runtime errors and ensures logical consistency, satisfying the KISS principle.

In alignment with REQ3 (Edit 07/04), the Slime uses a localized ConsumeBehaviour. This allows the slime to interact with the environment through Polymorphism. Because items like Apple and Cookies implement the Consumable interface, the slime can trigger their effects (like poisoning or healing) regardless of where the item is located. This effectively delegates the "effect" logic to the item itself, reducing coupling between the creature and the scrap it eats.

Finally, the Undead implementation reuses the Dependency Inversion Principle (DIP). The AttackBehaviour searches for any actor with the Ability.WORKER capability rather than checking for the concrete ContractedWorker class. This ensures that if new worker-type actors are added to Garbage Collection Inc. later, the Undead will automatically identify them as targets without requiring changes to its combat logic.

REQ4: The Facility Alarm System (HD Requirement)

A. Management of the Global Alarm State

Design 1: Use a static global boolean in the Application or EclipseNebula class that every entity polls every turn.

Pros	Cons
Very simple to implement and understand for basic triggers.	Violation of DIP: Low-level entities (Doors/Undead) become dependent on high-level world classes.
No need for additional manager classes or complex patterns.	Inefficiency: Every entity must manually check the state every turn, even if they don't respond to the alarm.
	Violation of SRP: High-level world classes should not be responsible for managing specific alarm timers and durations.

Design 2: Implement a Singleton AlarmManager utilizing the Observer Pattern with an AlarmListener interface.

Pros	Cons
High Extensibility (OCP): New consequences can be added by simply making a class implement AlarmListener. The AlarmManager remains unchanged.	Requires more initial boilerplate code to implement the subscription and notification logic.
Decoupling (DIP): The AlarmManager does not know which concrete classes it is notifying, it only knows it is talking to AlarmListeners.	Increased complexity in managing the lifecycle of subscribers.
Separation of Concerns (SRP): The AlarmManager is solely responsible for the alarm's state, while listeners handle their own specific reactions.	

B. Alarm Triggering Mechanism (Security Camera)

Implementation of a SecurityCamera class that extends NonPlayerCharacter and utilizes ScanningBehaviour to perform surveillance.

Pros	Cons
Reusability: Reuses the engine's Actor turn-taking logic and visibility mechanics to watch its surroundings.	Requires a stationary Actor that does nothing except surveillance, which might be less intuitive than a Ground tile.

Decoupling: The camera only triggers the alarm. It does not know what the consequences or listeners are.	
OCP & Hierarchy Integration: By extending NonPlayerCharacter, the camera uses the base priority loop rather than a hardcoded playTurn. This proves the NPC system is flexible enough for non-biological entities.	
Separation of Detection and Activation: Using ScanningBehaviour for detection and TriggerAlarmAction for activation follows SRP. The behavior finds the threat. The action executes the world-state change.	

C. Alarm Consequences - Hostile Behavior (The Chase)

Design 1: Modify the playTurn() of the Undead to include a pathfinding switch that activates when the alarm is on.

Pros	Cons
Direct implementation within the actor allows for quick prototyping.	Violation of SRP: The Actor class becomes cluttered with complex pathfinding algorithms.
No need for additional behavior classes.	Low Reusability: If a Security Drone is added later, the pathfinding logic must be duplicated or moved.

Design 2: Create a dedicated HuntBehaviour class (Strategy Pattern) that calculates the shortest path to the nearest conscious worker.

Pros	Cons
High Code Reuse (DRY): This behaviour can be given to any hostile entity (Undead, Slimes, etc.) without duplicating logic.	Computational Complexity: The findNearestWorker() method iterates over the entire GameMap range every turn. On very large maps, this could cause performance lag.
Prioritized Decision-Making: Using a priority (e.g., 10) ensures the "Chase" is more important than "Wander" (999), but less important than "Attack" (1), keeping the AI logical.	Manhattan Distance Limitation: The algorithm doesn't account for walls; a creature might get stuck trying to move "toward" a worker who is behind a solid wall.

D. The Alarm Clock Mechanism (Time Management)

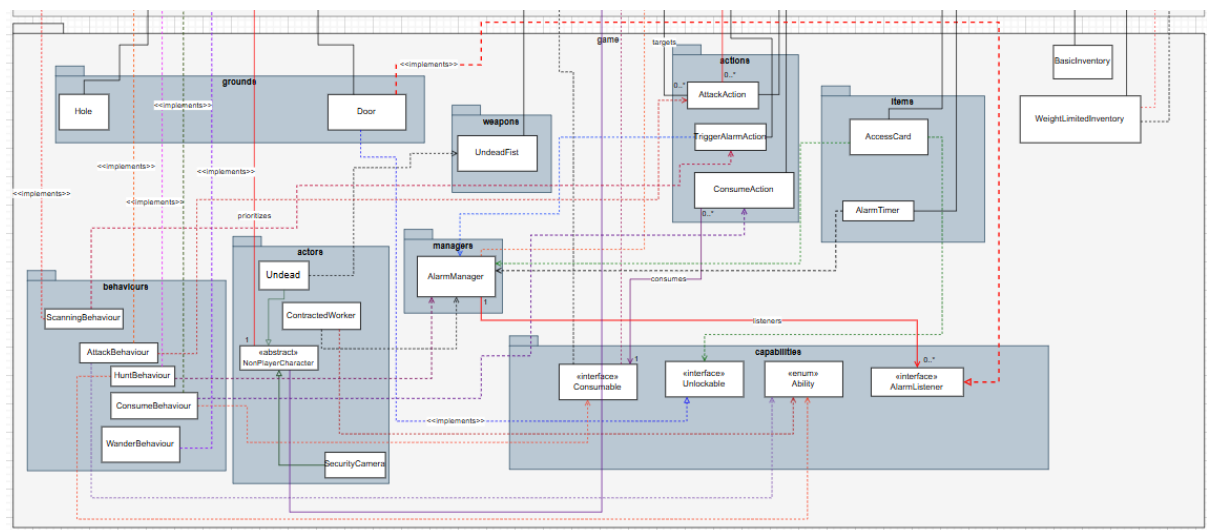
Design 1: Call AlarmManager.getInstance().tick() inside the playTurn() method of the ContractedWorker.

Pros	Cons
------	------

Simple; requires no extra map entities or classes.	Violation of DRY and SRP: If multiple workers exist, the alarm ticks down faster than intended. If no workers are conscious, the alarm stops ticking entirely.
--	--

Design 2: Create a stationary AlarmTimer Item placed at a fixed coordinate (e.g., 0,0) to call the manager's tick().

Pros	Cons
Multiplayer Fairness: Ensures the alarm counts down by Game Rounds, not Player Turns. This prevents the alarm from ending 5x faster just because 5 friends are playing together.	Requires a ⌚ item on the map which might be seen as an unnecessary map entity.



The diagram above shows the final design of requirement 4, which utilizes Design 2 for sections A, C and D.

To satisfy the HD requirement for a robust and scalable alarm system, I implemented the Observer Pattern. The AlarmManager serves as the Subject, maintaining a list of AlarmListener objects. This design follows the Dependency Inversion Principle (DIP) because the manager depends on an abstraction (AlarmListener) rather than concrete classes like Door or Undead. This makes the system "Open" for extension but "Closed" for modification (OCP). If Garbage Collection Inc. introduces new consequences, we simply implement the interface without changing the core manager logic.

The AlarmManager is implemented as a Singleton. This ensures that there is only one "Source of Truth" for the alarm state across the Moon facility, adhering to the Single Responsibility Principle (SRP). The AlarmTimer item acts as the bridge between the engine and the manager, calling the tick() method once per round to ensure the alarm duration is tracked accurately.

The AlarmTimer item's tick() method is invoked by the engine for every actor's turn, including unconscious or deceased workers who perform DoNothingAction. This design choice ensures the alarm duration is measured in game rounds rather than the number of surviving workers. If the alarm only ticked for conscious workers, a team with one surviving worker would experience an alarm lasting five times longer than a team with five workers for the same ALARM_DURATION value, creating unfair gameplay scaling. By ticking on every turn (dead or alive), the alarm consistently lasts exactly ALARM_DURATION rounds regardless of how many workers remain conscious.

While the Observer Pattern and Singleton Manager add architectural complexity, this design follows KISS by centralizing alarm logic in one place rather than scattering boolean checks across every entity. Furthermore, extending the SecurityCamera from NonPlayerCharacter ensures architectural consistency, allowing the camera to utilize the same prioritized decision-making loop as biological entities. If a new "Security Drone" were added in a future requirement, it would simply implement AlarmListener without modifying AlarmManager. The trade-off of increased boilerplate code (subscription management) is justified because REQ4 requires multiple consequences (chase and lockdown) that would otherwise require modifying each affected class individually. Similarly, HuntBehaviour was chosen over modifying Undead's playTurn() despite the performance cost of Manhattan Distance calculations each turn, because the behaviour can be reused by any future hostile entity without code duplication.

The triggering mechanism is handled by the SecurityCamera, which extends NonPlayerCharacter. By utilizing ScanningBehaviour, it performs a surveillance sweep of adjacent tiles via the inherited priority loop. This follows the Interface Segregation Principle (ISP) by checking only for the WORKER capability. Upon detection, the behavior returns a TriggerAlarmAction to notify the AlarmManager. This approach strictly adheres to SRP, as the behavior handles detection while the action handles world-state activation, keeping the trigger decoupled from the consequences.

The first consequence, the "Chase," is implemented via the HuntBehaviour. This behavior utilizes Polymorphism and the Strategy Pattern. When the alarm is active, the Undead overrides its wandering logic to move toward the nearest conscious worker using Manhattan Distance. Because this logic is encapsulated in a separate behavior class, it adheres to SRP and can be easily applied to future hostile entities.

The second consequence, the "Lockdown," is handled by the Door class implementing AlarmListener. When notified of an activation, the door enters an alarmLock state where canActorEnter() returns false, regardless of its previous state. This demonstrates Liskov Substitution Principle (LSP) as the Door maintains its contract as a Ground tile while adding specialized state-based restrictions that align with the high-security theme of the moon facility.